Step 3 — The Test-Coverage Reviewer, the Supervisor (Routing + Compile) — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and where the multi-agent value lives

Hint 1 — The idea, and the exact pieces

Finish the team: one more reviewer, plus the Supervisor that routes and compiles — the same Supervisor + `Command` shape from Project 4/Doc11.

- `src/agents/coverage_reviewer.py` — `run_coverage(files)`: flag a changed code file (`.py` / `.js` / `.ts` , not a test) when the diff has no test change at all.
- `src/agents/supervisor.py` — `choose_reviewers(files)` , `decide_next(state)` , `compile_review(findings)` , and the `supervisor` node (returns a `Command`).
- `src/state.py` — `ReviewState` (TypedDict) + `start_state(diff)` .
- `src/graph.py` — `build_graph(model, collection)` .

Hint 2 — The plan, and where the multi-agent value lives

```
choose_reviewers(files):           # route by file category (from categorize)
    code present  -> security, coverage
    code OR docs  -> style          (but NOT config-only)
decide_next(state): first reviewer in state["reviewers"] not in state["done"],
                    else "compile"
compile_review(findings): dedup by (category, message); sort by
                    {high:0, medium:1, low:2}; empty -> "no issues found."
```

The graph is `START -> supervisor` ; each reviewer node runs its reviewer and returns `Command(goto="supervisor", update={findings+=, done+=})` ; the supervisor sends control to the next pending reviewer, or to `compile -> END` .

The whole project exists to prove **routing** (a docs-only diff never pays for a security check) and **compiling** (one de-duplicated, severity-sorted comment, not three raw reports). Those are the two

tests that matter — `test_routing` and `test_compile`. The whole file set is in the Solution.